

The original artifact, Paint1, is a program designed to calculate how much paint is needed to fully coat one wall. It was created on April 18th, 2019, for my class IT 145 Foundation in Application Development. I chose this artifact for my portfolio because it is one of the very first projects I ever built, and I wanted to demonstrate my dedication to improving my existing skills, and my ability to make a simple program far more versatile.

This artifact, now called Paint2, is an improved version of the original program. It has become more dynamic, allowing users to calculate the gallons of paint needed to paint a whole room if needed, as well as including how many coats they intend to do. There is also proper error handling, which stops users from inputting negative values or strings without closing the program.

I met my planned course outcome, “Demonstrate an ability to use well-founded and innovative techniques, skills, and tools in computing practices for the purpose of implementing computer solutions that deliver value and accomplish industry-specific goals.” By building a more versatile program than my original, but I also would argue I met a separate, unintended outcome due to my error handling: “Develop a security mindset that anticipates adversarial exploits in software architecture and designs to expose potential vulnerabilities, mitigate design flaws, and ensure privacy and enhanced security of data and resources.”

As I was creating Paint2, I first focused on achieving the original functionality of the program. Since I was porting Paint1 over from JAVA to Python, I needed to ensure I could get it to do the exact same thing. After this, I focused on the enhancements, starting with the ability to calculate for more than one wall. I learned that not every function requires a class, since I initially tried to build my wall_count method using a class and then call it later, but it simplified

the program greatly by removing that completely and instead building the wall_count functionality right in the main program.

I faced quite a few challenges when it came to error handling. I wanted my code to be robust, and capable of handling all types of input without throwing an exception and stopping the program. It began with simpler steps of error handling, such as only running a module when a certain criterion was met. For example, only accepting wall widths IF the wall count was above 0. But then came the issue of string input. The program would just stop running if you input anything other than a numerical value, and through much trial and error I came across [this tutorial on error handling](#), where try and exception statements were used to validate input. This changed my program completely, and you could input as many strings as you wanted, and the program will continue to prompt you for the right information.